一.定义及分类

程序直接控制（查询）方式

基本思想

- CPU通过执行一段I/O程序来实现与I/O设备的数据交换，这是最原始、最简单的I/O控制方式。

两种主要方式

- 程序查询方式（又称为轮询方式，Polling）
  - 每次数据传送前，CPU必须主动查询I/O设备状态。
  - 属于有条件传送方式（I/O设备就绪后才操作）。
  - I/O程序通过与I/O接口中的命令寄存器DCR、状态寄存器DSR以及数据缓冲寄存器DBR进行数据交互实现设备控制和传输。

注意：
- 上述各寄存器的位宽、数目以及功能定义因设备和厂商而异，编写驱动程序时必须遵循具体约定。
- 不同类型的I/O设备，其程序查询的流程有所不同。

- 直接传送方式
  - 无需查询I/O设备状态，直接进行数据交互。
  - 属于无条件传送方式。
  - 可视为程序查询方式的特例，一般适用于非常简单的I/O设备（如开关、LED显示）。

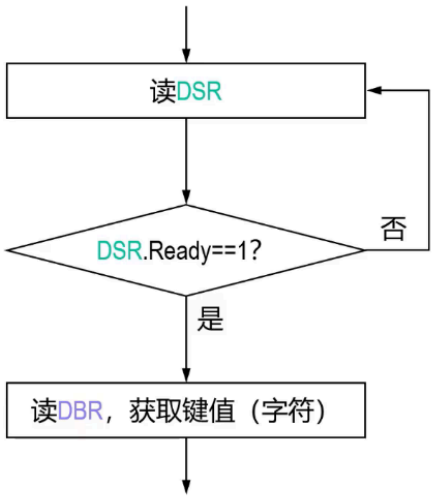
二.程序直接控制方式举例

2.1 简单IO的流程

程序直接控制（查询）方式 —— 举例

简单的输入设备：键盘

- 没有命令寄存器DCR
- 当按下按键时，键盘控制器执行以下操作：
  - 将被按下的按键的键值（ASCII）存放在键盘的数据缓冲寄存器DBR中。
  - 同时将键盘的状态寄存器DSR中的“Ready”标志位置为1，表示当前有按键数据。
- 当CPU需要从键盘获取按键数据时，则通过总线读取DSR寄存器，以便查询DSR中的Ready标志位：
  - 若为0，则继续查询直至该位为1；
  - 若为1，则通过总线读取DBR中的键值，完成操作后DSR中的Ready标志位自动清零。

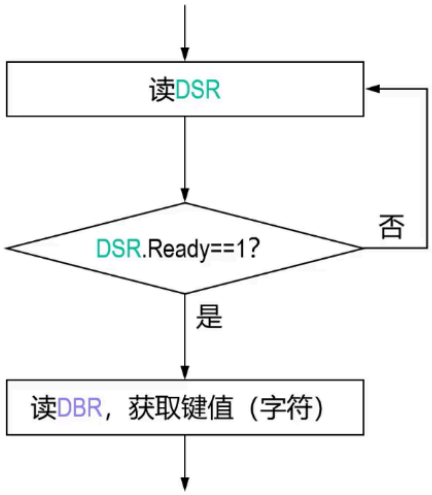


如果需要连续获取多个按键，需要给上述流程加上循环控制。

对应的MIPS指令(统一编址)

程序直接控制（查询）方式 —— 举例

简单的输入设备：键盘



MIPS架构中I/O接口的编址采用统一编址（内存映射编址）方式，使用访存指令访问，假设：

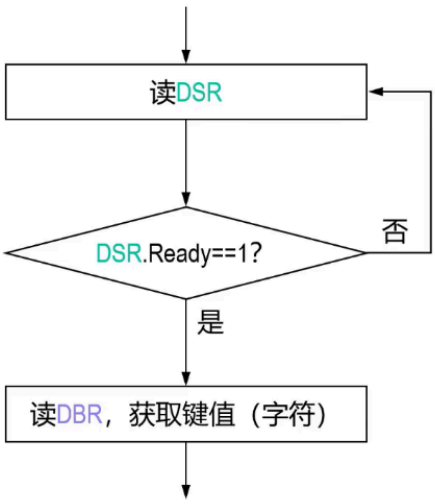
- 键盘设备中的8位设备状态寄存器DSR的内存映射地址为0xffff0000，最低位为Ready标志位。
- 键盘设备中的8位数据缓冲寄存器DBR的内存映射地址为0xffff0004。

```
        lui  $t1, 0xffff0000    # 载入键盘状态寄存器DSR的内存映射地址到$t1
keyPoll: lbu  $t4, ($t1)        # 载入DSR的值到$t4
        andi $t4, $t4, 1        # 获取DSR的Ready标志位（最低位）
        beq  $t4, $0, keyPoll   # 若Ready==0，则跳转到keyPoll处继续查询
        lui  $t1, 0xffff0004    # 载入键盘数据缓冲寄存器DBR的内存映射地址到$t1
        lbu  $s2, ($t1)         # 读取DBR中的字符数据（键值）到$s2
```

对应的x86指令(独立编址)

程序直接控制（查询）方式 —— 举例

简单的输入设备：键盘

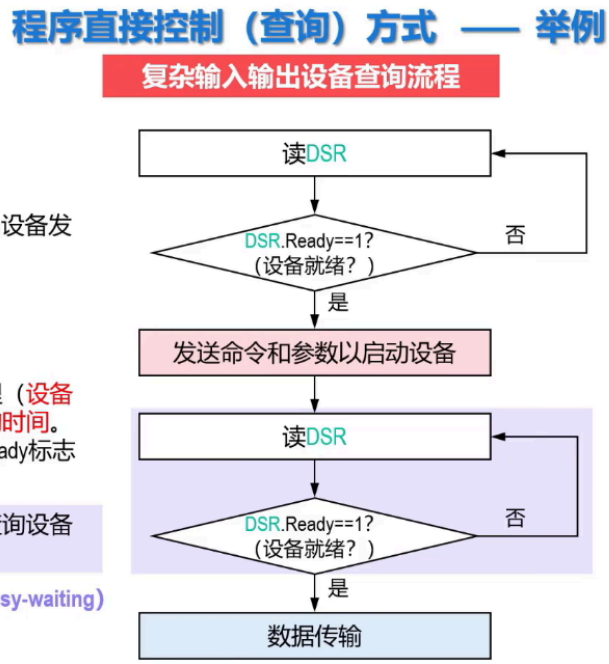


x86架构中I/O接口的编址采用独立编址（端口映射编址）方式，使用专用的I/O指令访问，假设：

- 键盘设备中的8位设备状态寄存器DSR的端口地址为0x0000，最低位为Ready标志位。
- 键盘设备中的8位数据缓冲寄存器DBR的端口地址为0x0004。

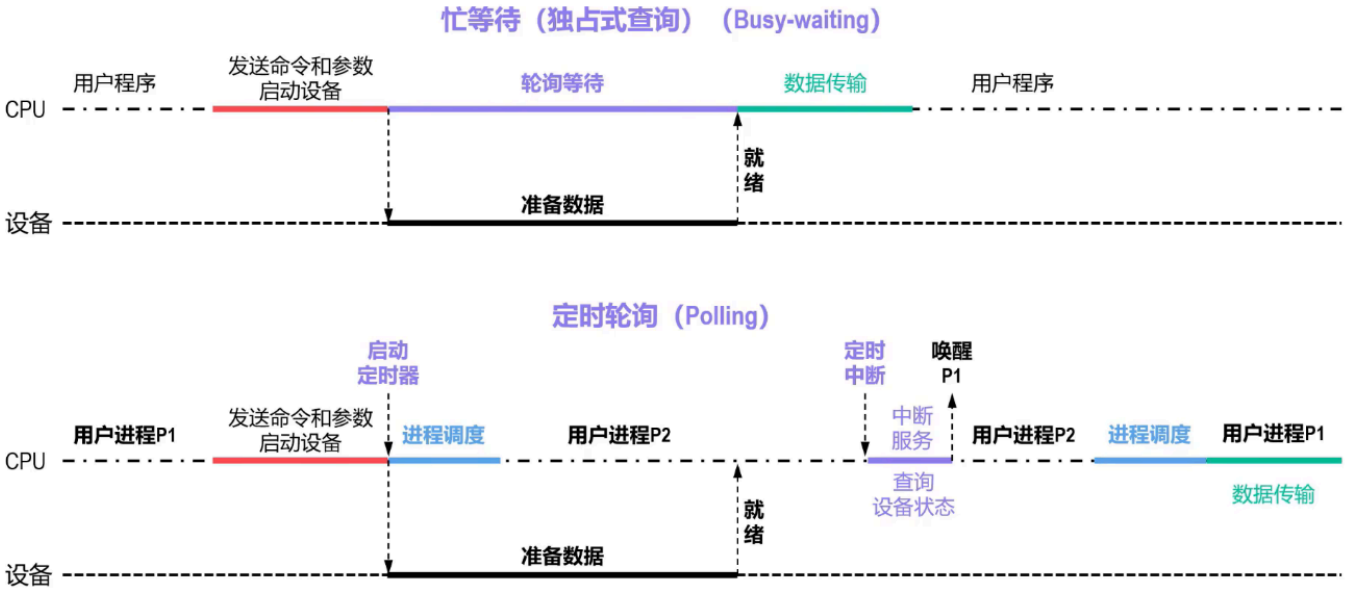
```
        MOV  DX, 0x0000     # 载入键盘状态寄存器DSR的端口地址到DX
keyPoll: IN   AL, DX        # 载入DSR的值到AL
        TEST AL, 1          # 测试DSR的Ready标志位（最低位）
        JZ   keyPoll        # 若Ready==0，则跳转到keyPoll处继续查询
        MOV  DX, 0x0004     # 载入键盘数据缓冲寄存器DBR的端口地址到DX
        IN   AL, DX         # 读取DBR中的字符数据（键值）到AL
```

2.2 复杂IO的流程



- 若发现设备不正常，也可直接向设备发送复位命令。
- 设备收到命令后立刻准备或处理（设备准备阶段），这可能需要较长的时间。设备准备好后，会将DSR中的Ready标志位置1。
- CPU在启动设备后就开始持续查询设备状态（轮询）。
  - 忙等待（独占式查询）（Busy-waiting）
  - 定时轮询（Polling）

忙等待与定时轮询



关于轮询的无效耗时

- 轮询等待浪费了大量CPU时间。
- 一般在单任务操作系统中可采用独占式查询，因为CPU在轮询等待过程中不需要执行其他任务。
- 对于超高速设备，由于设备响应快，相应的轮询等待时间短，独占式查询也可以接受。

定时轮询的优点

- 设备准备数据的过程中，**CPU可执行其他任务。**
- **有效避免了轮询等待对CPU时间的大量浪费**，但中断服务还需要占用一定的CPU时间。
- 适用于**外部设备自身没有中断请求功能**的**多任务操作系统。**

关于定时时间间隔的取舍

- **定时时间间隔**的选择是关键问题：
  - **太短**：中断频繁，进而频繁进行**中断服务**，**开销浪费大。**
  - **太长**：I/O设备的**数据可能得不到及时处理。**
- 为了**提高CPU利用率**，并且**确保I/O设备的数据得到及时处理**而不丢失，需要**合理设置定时时间间隔。**
  - 例如：键盘设备和鼠标设备的输入速度比CPU慢得多，因此，只要以比人的反应速度快的频率进行定时轮询，就可以确保设备数据不丢失。

关于不同应用场景下的方式选择

- **慢速设备**比较适合**定时轮询**，而快速设备的定时轮询开销大。
- **程序直接控制（查询）方式**的I/O接口设计简单，容易实现，常见于**早期单任务操作系统**中。
- **现代多任务操作系统在多任务引导成功之前**，必须使用**程序直接控制（查询）方式**与I/O设备进行数据交互（如键盘、磁盘、显示器等都必须支持程序直接控制方式）。